

Backend Documentation - College Event Portal

1. Overview

This is the backend server for the College Event Portal, built with **Node.js**, **Express**, and **MongoDB (Mongoose)**. It handles user authentication, event management, approval workflows, announcements, and recruitment processes.

2. Project Structure

- **config/**: Configuration files (e.g., Database connection).
 - **controllers/**: Business logic for handling requests.
 - **models/**: Mongoose schemas defining data structure.
 - **routes/**: API route definitions mapping URLs to controllers.
 - **middleware/**: Interceptors for authentication, error handling, etc.
 - **utils/**: Helper functions (JWT generation, validation).
 - **server.js**: Entry point of the application.
-

3. Database Models (Schemas)

User (`models/User.js`)

Represents a user in the system.

- **Fields:**
 - `name`: String, required.
 - `email`: String, required, unique.
 - `password`: String, required (hashed).
 - `phone`: String, 10 digits.
 - `branch`: String, required (e.g., CSE, IT).
 - `year`: String, enum [`First Year`, `Second Year`, `Third Year`]. Required for students.
 - `role`: Enum [`student`, `faculty`, `admin`]. Default: `student`.
 - `isApproved`: Boolean. `true` for students, `false` for faculty (requires admin approval), `true` for admin.
- **Key Logic:**
 - Pre-save hook hashes password.
 - `comparePassword` method checks credentials.

Event (`models/Event.js`)

Represents an approved, official event.

- **Fields:**
 - `title`, `description`, `date`, `venue`: Required details.
 - `branch`: Target branch ("ALL" or specific).

- **createdBy**: Reference to **User** (Faculty/Admin).

EventRequest (**models/EventRequest.js**)

A proposal for an event submitted by a **Student**.

- **Fields:**
 - **title, description, date, venue, branch**.
 - **requestedBy**: Reference to **User** (Student).
 - **status**: Enum [**pending, approved, rejected**]. Default: **pending**.
 - **reviewComment**: Reason for approval/rejection.
 - **reviewedBy**: Reference to **User** (Faculty/Admin).
 - **event**: Reference to created **Event** (if approved).

Announcement (**models/Announcement.js**)

General updates posted by faculty/admins.

- **Fields:**
 - **title, message**.
 - **branch**: Target audience (e.g., "all", "IT").
 - **createdBy**: Reference to **User**.

Recruitment (**models/Recruitment.js**)

Volunteer/Organizer recruitment posts for events.

- **Fields:**
 - **event**: Reference to the **Event**.
 - **title, description**.
 - **roleType**: Enum [**volunteer, anchor, coordinator, technical, other**].
 - **branch**: Target branch.
 - **status**: Enum [**open, closed**].
 - **createdBy**: Reference to **User**.
 - **applicants**: Array of sub-documents:
 - **student**: Reference to **User**.
 - **note**: Application message.
 - **status**: [**applied, selected, rejected**].

4. Authentication & Authorization

Middleware

- **authMiddleware**: Verifies JWT from **Authorization: Bearer <token>** header. Attaches user to **req.user**.
- **roleMiddleware(...roles)**: Restricts access based on **req.user.role**.

Auth Flow

1. Registration:

- Students get a token immediately (if valid).
- Faculty get a success message but **no token** until Admin approves them.

2. Login:

- Checks email/password.
- Checks **isApproved**. If false, login is denied.

3. Google Auth:

- Supports Google Sign-In.
- Auto-creates accounts (Students auto-approved, Faculty pending).

5. API Routes & Controllers

Auth (/api/auth)

Method	Endpoint	Access	Description
POST	/register	Public	Register new user.
POST	/login	Public	Login active user.
POST	/google	Public	Google Login/Signup.
POST	/google/check	Public	Check if Google user exists.

Users (/api/users)

Method	Endpoint	Access	Description
GET	/me	Auth	Get current logged-in user details.

Events (/api/events)

Method	Endpoint	Access	Description
GET	/	Public	Get all events.
GET	/:id	Public	Get single event details.
POST	/	Faculty/Admin	Create a new event.
PUT	/:id	Faculty/Admin	Update an event.
DELETE	/:id	Admin	Delete an event.

Event Requests (/api/event-requests)

Method	Endpoint	Access	Description
POST	/	Student	Submit a new event proposal.
GET	/my	Student	View my submitted requests.

Method	Endpoint	Access	Description
GET	/	Faculty/Admin	View all requests (Faculty sees only their branch options).
GET	/:id	Faculty/Admin	View request details.
PATCH	/:id/approve	Faculty/Admin	Approve request -> Creates Event .
PATCH	/:id/reject	Faculty/Admin	Reject request.

Announcements (/api/announcements)

Method	Endpoint	Access	Description
GET	/	Public	View all announcements.
GET	/:id	Public	View single announcement.
POST	/	Faculty/Admin	Post new announcement.
PUT	/:id	Faculty/Admin	Edit announcement.
DELETE	/:id	Admin	Delete announcement.

Recruitments (/api/recruitments)

Method	Endpoint	Access	Description
GET	/	Public	View open recruitment posts.
GET	/:id	Public	View details.
POST	/:id/apply	Student	Apply for a role.
POST	/	Faculty/Admin	Create recruitment post.
GET	/:id/applicants	Faculty/Admin	View students who applied.
PUT	/:id	Faculty/Admin	Update post.
DELETE	/:id	Admin	Delete post.

Admin (/api/admin)

Method	Endpoint	Access	Description
GET	/users	Admin	List all system users.
GET	/users/:id	Admin	Get user details.
PATCH	/users/:id/role	Admin	Change user role.
PUT	/users/:id/approve	Admin	Approve pending user (Faculty).
DELETE	/users/:id	Admin	Delete user.

6. Utils & Helpers

- `generateToken.js`: Creates JWTs expiring in 7 days.
- `validateObjectId.js`: Middleware to ensure MongoDB IDs are valid before querying.
- `apiResponse.js`: Standardizes API responses (`success: true/false`, `message`, `data`).

7. Error Handling

- Global Error Handler installed in `server.js`.
- Catches exceptions and sends clean JSON error responses.
- Handles standard MongoDB validation errors (e.g., duplicates, missing fields).